

93kbench Tutorial

93kbench
February 17, 2011

Contents

1	93kbench Tutorial	1
1.1	Introduction	1
1.2	Device Description	2
1.2.1	DUT pin definitions	2
1.3	Running 93kbench	3
1.3.1	Create setup file	3
1.3.2	Compile the DUT model with the Test Bench	6
1.3.3	Run simulation	7
1.3.4	Simulation results	8

Chapter 1

93kbench Tutorial

1.1 Introduction

1. **93kbench concept**

93kbench software provides back annotate from the production test program to the design environment . Using that link, provides the designer a tester like environment to verify the operation of the test program.

The Inputs to 93kbench are ATE test files of Verigy 93000 testers.

The Output is design simulation file (Verilog VCD dump file) with verilog design model of virtual tester.

2. **Tutorial goal**

The purpose of this tutorial is to familiarize you with 93kbench software. After completing this tutorial you will be able to convert 93000 ATE file to virtual tester using VCD file.

1.2 Device Description

1.2.1 DUT pin definitions

Below is a list of the direction of device signals and IO control signals as defined in the device specification:

Pin	Direction	I/O control
A0	Input	-
A1	Input	-
CLK0	Input	-
CLK1	Input	-
CLK2	Input	-
CS_	Input	-
D0	Inout	Controlled by D_dir
D1	Inout	Controlled by D_dir
D2	Inout	Controlled by D_dir
D3	Inout	Controlled by D_dir
D4	Inout	Controlled by D_dir
D5	Inout	Controlled by D_dir
D6	Inout	Controlled by D_dir
D7	Inout	Controlled by D_dir
GATE0	Input	-
GATE1	Input	-
GATE2	Input	-
OUT0	Output	-
OUT1	Output	-
OUT2	Output	-
RD_	Input	-
WR_	Input	-
D_dir	Input	-

Table 1.1: Device Direction definitions

D_dir is the Internal IO control pin for D0-D7 signals.

Note:

When D_dir IO control signal value = 1 Inout signal direction is an Input.

When D_dir IO control signal value = 0 Inout signal direction is an Output.

1.3 Running 93kbench

Change directory to /93kbench/ Under this directory you will find the following files:

1. 93K_ATE_program/
2. vtrun1
3. setup_93k.py
4. Device_files/

1.3.1 Create setup file

In order to generate template of setup file and edit it run the following command:

```
>93kbench -template
```

You should get the following log:

```
The tool will generate template setup file in  
'./93k_setup_template.py' and silently exit.
```

Here we have done the setup file for you just run :

```
>93kbench -setup setup_93k.py
```

You should get the following log:

```

=====
= Verigy 93000 -> STIL =
=====

Reading Verigy 93000 Data Files from '93K_ATE_program/'...
 93K_ATE_program//testflow/Simple.tf
 93K_ATE_program//vectors/Simple.pmfl
 93K_ATE_program//configuration/Simple_pins.cha
 93K_ATE_program//levels/Simple.lvl
 93K_ATE_program//timing/Simple.tim
Done [reading verigy files]

Collecting Labels Info...
 93K_ATE_program/vectors/patterns/WR_RD_Mode0.bin1
 93K_ATE_program/vectors/patterns/Burst.burst
Done [labels info].

Analyzing Test Flow...
Done [analyzing flow]

Translating Labels...

Label 'Burst':
Loading '93K_ATE_program/vectors/patterns/Burst.burst'...
  Translating...
Status: OK

Label 'WR_RD_Mode0':
Loading '93K_ATE_program/vectors/patterns/WR_RD_Mode0.bin1'...
  Translating...
Status: OK

Done [Labels].

TestSuite 'setup':
SPECSET   1 "Default"
WAVETBL   "WFT"
EQNSESET  1 "WFT"
TIMINGSET 1 "WFT"
EQNSESET  1 "default"
LEVELSET  1 "default"
SPECSET   1 "Default"
SQLB      "Burst"
Status: OK

=== Translation Report ===
setup ..... OK
=====
Total: 1 Passed: 1 Failed: 0

=====
= STIL -> VTB =
=====

Activating Testbench Generator...

File: 'vt/stil/setup.stil'
Pattern: WR_RD_Mode0

Done [Testbench Generator].

```

Look at *setup_93k.py*.

We have done the following edit:

1. `_93k_testdir = '93K_ATE_program/'` - link to the directory of 93K tester files.
2. `_93k_testflow = 'Simple.tf'` - link to the test flow file, you can see this file under testflow directory .
3. `_93k_testsuite = 'setup'` - link to the test suite, you can see it in the file Simple.tf.
4. `dut_name = "I8253"` - the dut top level name.

1.3.2 Compile the DUT model with the Test Bench

After running 93kbench a directory *vt* should be generated.

Change directory to *vt* directory . Now you need to change the file *vtrun1* with the following changes:

1. change the first line from verilog to vcsi.
2. add in the end of the file link to the Device design files.

```

1  verilog \ ---> Change to vcsi
2  test.hv \
3  board.hv \
4  ../sw/ate2stil_1.1.5_Linux/etc/verilog/vtw_wfgdef.hv \
5  ../sw/ate2stil_1.1.5_Linux/etc/verilog/vtw_stddef.hv \
6  ../sw/ate2stil_1.1.5_Linux/etc/verilog/vtw_logger.hv \
7  ../sw/ate2stil_1.1.5_Linux/etc/verilog/vtw_alldef.hv \
8  ../sw/ate2stil_1.1.5_Linux/etc/verilog/clockgen.v \
9  ../sw/ate2stil_1.1.5_Linux/etc/verilog/vtw_logger.v \
10 ../ate2stil_1.1.5_Linux/etc/verilog/vtw_wfg.v \
11 WR_RD_Mode0_timing.v \
12 WR_RD_Mode0.v \
13 WR_RD_Mode0_timing_control.v \
14 test_wrapper.v \
15 tester.v \
16 monitor.v \
17 ../Device_files/CNTREG.v \    --> Add link to device model
18 ../Device_files/DOWNCNTR.v \ --> Add link to device model
19 ../Device_files/I8253.v \    --> Add link to device model
20 ../Device_files/MODEREG.v \  --> Add link to device model
21 ../Device_files/OUTCTRL.v \  --> Add link to device model
22 ../Device_files/OUTLATCH.v \ --> Add link to device model
23 ../Device_files/READ.v      --> Add link to device model
24

```

In the main directory there is *vtrun1* file, you can look at it to see how it should look like, do not copy this file because links can be different.

1.3.3 Run simulation

In order to run the simulation run:

```
source vtrun1
```

You should get the following log:

```
.....
.....
This program is proprietary and confidential information of Synopsys Inc.
and may be used and disclosed only as authorized in a license agreement
controlling such use and disclosure.

Parsing design file 'test.hv'
Parsing design file 'board.hv'
.....
.....

Parsing design file 'WR_RD_Mode0_timing_control.v'
Parsing design file 'test_wrapper.v'
Parsing design file 'tester.v'
Parsing design file 'monitor.v'
Parsing design file '../Device_files/CNTREG.v'
Parsing design file '../Device_files/DOWNCNTR.v'
Parsing design file '../Device_files/I8253.v'
Parsing design file '../Device_files/MODEREG.v'
Parsing design file '../Device_files/OUTCTRL.v'
Parsing design file '../Device_files/OUTLATCH.v'
Parsing design file '../Device_files/READ.v'
Top Level Modules:
    test_wrapper
TimeScale is 1 ps / 1 ps
Notice: Ports coerced to inout, use -notice for details
Starting vcs inline pass...
3 modules and 0 UDP read.
However, due to incremental compilation, no re-compilation is necessary.
if [ -x ../simv ]; then chmod -x ../simv; fi
gcc -o ../simv 8E6T_1_d.o MPUG_1_d.o 3sNO_1_d.o SIM_1.o 5NrI_d.o 5NrIB_d.o
/opt/vcs/vcs7.2/gui/virsim/linux/vcdplus/vcs7_2/libvirsim.a
/opt/vcs/vcs7.2/linux/lib/libvcsnew.so -ldl -lc -lm -ldl
../simv up to date
CPU time: .457 seconds to compile + .141 seconds to link
```

Now what is left to do is running the simulation:

```
./simv
```

1.3.4 Simulation results

The log for the simulation run:

```

Compiler version VCSi 7.2; Runtime version VCSi 7.2;  Mar 17 13:13 2010

Opening log files : vtw.log,vtw.fail,vtw.warn.
[ VTVECINF ][ vector information ][ pattern WR_RD_Mode0 ]
[ vector #0 ][ wft WFT ][ time 0 ]
[ VTDRVCNT ][ contention between device and tester ][ signal D0 ][ time 0 ]
[ VTDRVCNT ][ contention between device and tester ][ signal D1 ][ time 0 ]
[ VTDRVCNT ][ contention between device and tester ][ signal D2 ][ time 0 ]
[ VTDRVCNT ][ contention between device and tester ][ signal D3 ][ time 0 ]
[ VTDRVCNT ][ contention between device and tester ][ signal D4 ][ time 0 ]
[ VTDRVCNT ][ contention between device and tester ][ signal D5 ][ time 0 ]
[ VTDRVCNT ][ contention between device and tester ][ signal D6 ][ time 0 ]
[ VTDRVCNT ][ contention between device and tester ][ signal D7 ][ time 0 ]
[ VTVECINF ][ vector information ][ pattern WR_RD_Mode0 ]
[ vector #1 ][ wft WFT ][ time 400000 ]
[ VTVECINF ][ vector information ][ pattern WR_RD_Mode0 ]
[ vector #2 ][ wft WFT ][ time 800000 ]
[ VTVECINF ][ vector information ][ pattern WR_RD_Mode0 ]
[ vector #3 ][ wft WFT ][ time 1200000 ]
[ VTVECINF ][ vector information ][ pattern WR_RD_Mode0 ]
[ vector #4 ][ wft WFT ][ time 1600000 ]
[ VTVECINF ][ vector information ][ pattern WR_RD_Mode0 ]
[ vector #5 ][ wft WFT ][ time 2000000 ]
.....
.....

[ VTVECINF ][ vector information ][ pattern WR_RD_Mode0 ]
[ vector #87 ][ wft WFT ][ time 34800000 ]
[ VTVECINF ][ vector information ][ pattern WR_RD_Mode0 ]
[ vector #88 ][ wft WFT ][ time 35200000 ]
[ VTVECINF ][ vector information ][ pattern WR_RD_Mode0 ]
[ vector #89 ][ wft WFT ][ time 35600000 ]

-----
Verilog TestBench completed, see log files vtw.log,vtw.fail,vtw.warn.
Status : simulation passed.
Total number of compare operations executed : 621.
-----

finish at simulation time          36000000
          V C S   S i m u l a t i o n   R e p o r t
Time: 36000000 ps
CPU Time:      0.220 seconds;          Data structure size:   0.4Mb

```

As you can see in this simulation log all the compare results match to the Device output. So the STIL file behavior is matching to our Device behavior.

For simulation results see the following files:

1. **vtw.log** : Logfile for the run.
2. **vtw.fail** : In case there were failures, see this file.
3. **vtw.warn** : In case there were warnings, see this file.